

后缀数组简介

一些约定

字符串相关的定义请参考 [字符串基础](#)。

字符串下标从 0 开始。

字符串 s 的长度为 $|s|$ 。

"后缀"代指以第 i 个字符开头的后缀，存储时用 $sa[i]$ 代表字符串 s 的后缀。

后缀数组是什么？

后缀数组 (Suffix Array) 主要关系到两个数组： sa 和 $rank$ 。

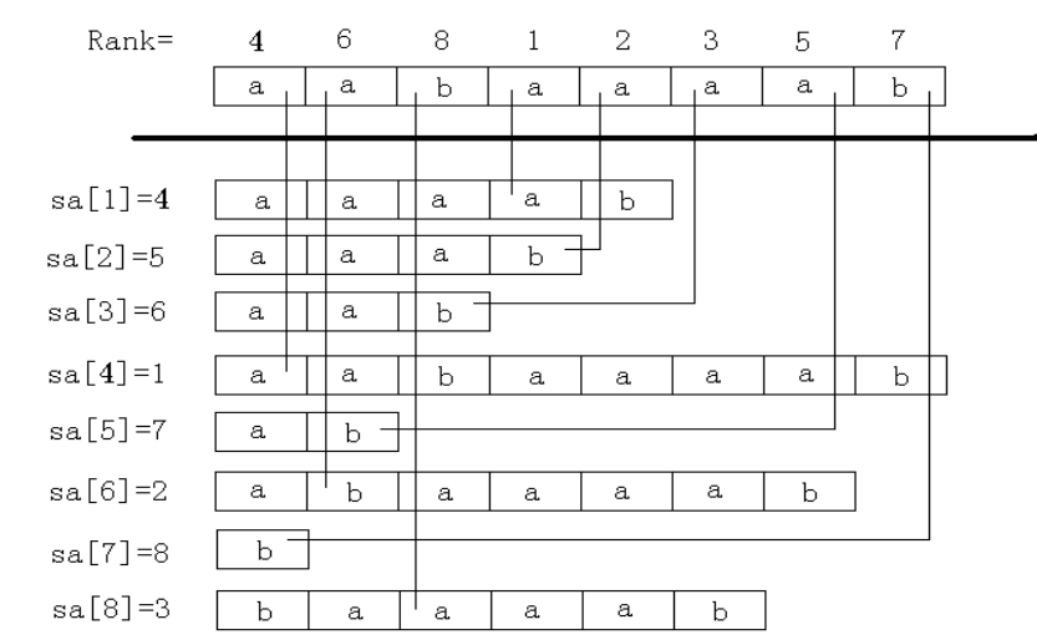
其中， $sa[i]$ 表示将所有后缀排序后第 i 小的后缀的编号，也是所说的后缀数组，后文也称编号数组；

$rank[i]$ 表示后缀 $s[i..n]$ 的排名，是重要的辅助数组，后文也称排名数组。

这两个数组满足性质： $rank[sa[i]] = i$ 。

解释

后缀数组示例：



后缀数组怎么求？

$O(n^2 \log n)$ 做法

相信这个做法大家还是能自己想到的：将盛有全部后缀字符串的数组进行 `sort` 排序，由于排序进行 n 次字符串比较，每次字符串比较要 $\log n$ 次字符比较，所以这个排序是 $O(n^2 \log n)$ 的时间复杂度。

$O(n \log^2 n)$ 做法

这个做法要用到倍增的思想。

首先对字符串 s 的所有长度为 1 的子串，即每个字符进行排序，得到排序后的编号数组 sa 和排名数组 $rank$ 。

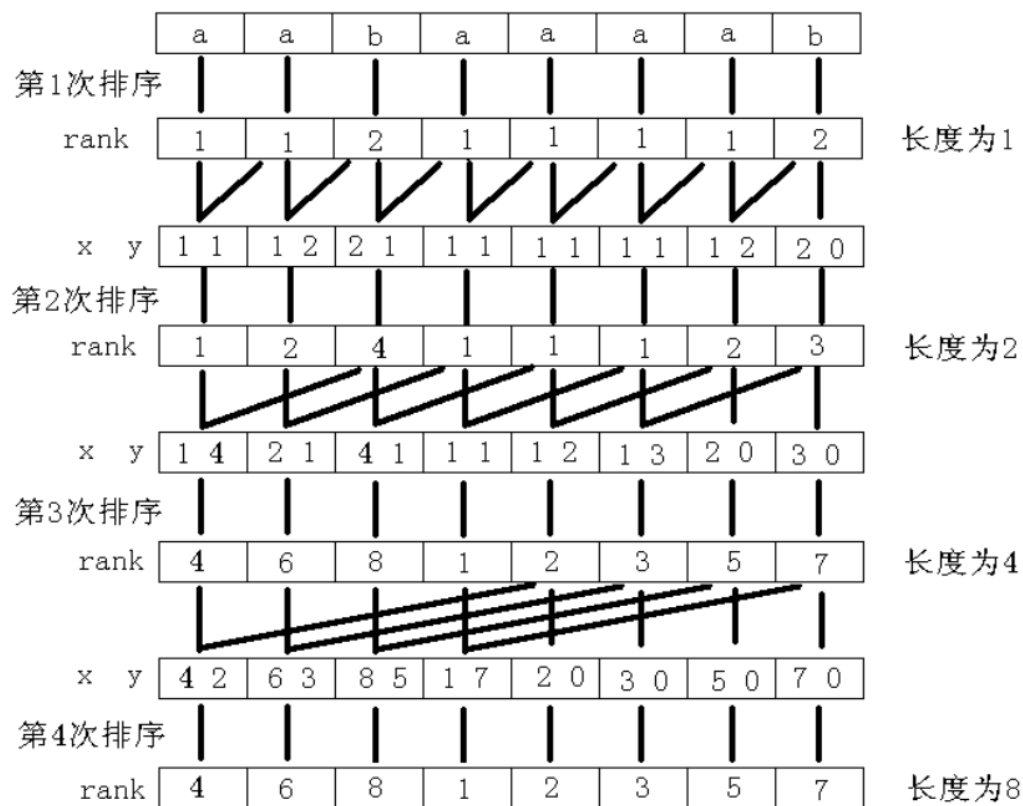
倍增过程：

- 用两个长度为 1 的子串的排名，即 $rank[i]$ 和 $rank[i+1]$ ，作为排序的第一第二关键字，就可以对字符串 s 的每个长度为 2 的子串：进行排序，得到 sa 和 $rank$ ；
- 之后用两个长度为 2 的子串的排名，即 $rank[i]$ 和 $rank[i+2]$ ，作为排序的第一第二关键字，就可以对字符串 s 的每个长度为 4 的子串：进行排序，得到 sa 和 $rank$ ；

3. 以此倍增，用长度为 k 的子串的排名，即 $rank_k$ 和 $rank_{k+1}$ ，作为排序的第一第二关键字，就可以对字符串 s 的每个长度为 k 的子串 进行排序，得到 $rank_{k+1}$ 和 $rank_{k+2}$ 。其中，类似字母序排序规则，当 $rank_k = rank_{k+1}$ 时，视为无穷小；
4. 即是子串 s 的排名，这样当 $k = n$ 时，得到的编号数组 $rank_n$ ，也就是我们需要的后缀数组。

过程

倍增排序示意图：



显然倍增的过程是，而每次倍增用 `sort` 对子串进行排序是，而每次子串的比较花费 $O(k)$ 次字符比较；

除此之外，每次倍增在 `sort` 排序完后，还有额外的 $O(n \log n)$ 的时间复杂度的，更新 $rank$ 的操作，但是相对于 $O(n^2 \log n)$ 被忽略不计；

所以这个算法的时间复杂度就是 $O(n \log n)$ 。

► 实现

$O(n \log n)$ 做法

在刚刚的 $O(n^2 \log n)$ 做法中，单次排序是 $O(n \log n)$ 的，如果能 $O(n)$ 排序，就能 $O(n \log n)$ 计算后缀数组了。

前置知识：[计数排序](#)，[基数排序](#)。

由于计算后缀数组的过程中排序的关键字是排名，值域为 $[0, n-1]$ ，并且是一个双关键字的排序，可以使用基数排序优化至 $O(n \log n)$ 。

► 实现

一些常数优化

如果你把上面那份代码交到 [LOJ #111: 后缀排序](#) 上：

▸ 测试点 #1	✓ Accepted	得分: 100	用时: 20 ms	内存: 11900 KiB
▸ 测试点 #2	✓ Accepted	得分: 100	用时: 27 ms	内存: 12004 KiB
▸ 测试点 #3	✓ Accepted	得分: 100	用时: 27 ms	内存: 11900 KiB
▸ 测试点 #4	✓ Accepted	得分: 100	用时: 37 ms	内存: 12112 KiB
▸ 测试点 #5	✓ Accepted	得分: 100	用时: 36 ms	内存: 12112 KiB
▸ 测试点 #6	✓ Accepted	得分: 100	用时: 103 ms	内存: 13200 KiB
▸ 测试点 #7	✓ Accepted	得分: 100	用时: 111 ms	内存: 13248 KiB
▸ 测试点 #8	⌚ Time Limit Exceeded	得分: 0	用时: 4018 ms	内存: 24576 KiB
▸ 测试点 #9	⌚ Time Limit Exceeded	得分: 0	用时: 4014 ms	内存: 24576 KiB
▸ 测试点 #10	⌚ Time Limit Exceeded	得分: 0	用时: 4018 ms	内存: 24576 KiB

这是因为，上面那份代码的常数的确很大。

第二关键字无需计数排序

思考一下第二关键字排序的实质，其实就是把超出字符串范围（即 `w`）的 `id` 放到 数组头部，然后把剩下的依原顺序放入：

```
1 int cur = 0;
2 for (int i = n - w + 1; i <= n; i++) id[++cur] = i;
3 for (int i = 1; i <= n; i++)
4   if (sa[i] > w) id[++cur] = sa[i] - w;
```

优化计数排序的值域

每次对 `id` 进行更新之后，我们都计算了一个 `cur`，这个 `cur` 即是 `id` 的值域，将值域改成它即可。

若排名都不相同可直接生成后缀数组

考虑新的 `id` 数组，若其值域为 `cur` 那么每个排名都不同，此时无需再排序。

► 实现

$O(n)$ 做法

在一般的题目中，常数较小的倍增求后缀数组是完全够用的，求后缀数组以外的部分也经常有 $O(n \log n)$ 的复杂度，倍增求解后缀数组不会成为瓶颈。

但如果遇到特殊题目、时限较紧的题目，或者是你想追求更短的用时，就需要学习 求后缀数组的方法。

SA-IS

可以参考 [诱导排序与 SA-IS 算法](#)，另外它的 [评论页面](#) 也有参考价值。

DC3

可以参考[\[2009\] 后缀数组——处理字符串的有力工具 by 罗穗骥](#)。

后缀数组的应用

寻找最小的循环移动位置

将字符串 `s` 复制一份变成 `ss` 就转化成了后缀排序问题。

例题：[「JSOI2007」字符加密](#)。

在字符串中找子串

任务是在线地在主串 `s` 中寻找模式串 `t`。在线的意思是，我们已经预先知道知道主串 `s`，但是当且仅当询问时才知道模式串 `t`。我们可以先构造出 `s` 的后缀数组，然后查找子串 `t`。若子串 `t` 在 `s` 中出现，它必定是 `s` 的一些后缀的前缀。因为我们已经将所有后缀排序了，我们可以通过在 `sa` 数组中二分 `t` 来实现。比较子串 `t` 和当前后缀的时间复杂度为 $O(|t|)$ ，因此找子串的时间复杂度为 $O(n \log |t|)$ 。注意，如果该子串在 `s` 中出现了多次，每次出现都是在 `sa` 数组中相邻的。因此出现次数可以通过再次二分找到，输出每次出现的位置也很轻松。

从字符串首尾取字符最小化字典序

例题：[「USACO07DEC」Best Cow Line](#)。

题意：给你一个字符串，每次从首或尾取一个字符组成字符串，问所有能够组成的字符串中字典序最小的一个。

- ▶ 题解
- ▶ 参考代码

height 数组

LCP（最长公共前缀）

两个字符串 s 和 t 的 LCP 就是最大的 k 使得 $s[1:k] = t[1:k]$ 。

下文中以 h_i 表示后缀 $s[i:n]$ 和后缀 $t[i:n]$ 的最长公共前缀（的长度）。

height 数组的定义

h_i ，即第 i 名的后缀与它前一名的后缀的最长公共前缀。

可以视作 $h_i = \text{LCP}(s[i:n], s[i-1:n])$ 。

$O(n)$ 求 height 数组需要一个引理

▼ 证明

当 $i = 1$ 时，上式显然成立（右边小于等于 h_1 ）。

当 $i > 1$ 时：

根据 h_{i-1} 的定义，有 $h_{i-1} = \text{LCP}(s[i-1:n], s[i-2:n])$ 。

既然后缀 $s[i-1:n]$ 和后缀 $s[i-2:n]$ 有长度为 h_{i-1} 的最长公共前缀，

那么不妨用 k 来表示这个最长公共前缀。（其中 k 是一个字符， $s[i-1:n]$ 是长度为 $n-i+1$ 的字符串，非空）

那么后缀 $s[i:n]$ 可以表示为 $k + s[i+k:n]$ ，后缀 $s[i-1:n]$ 可以表示为 $k + s[i-1+k:n]$ 。（ k 可能为空串， $s[i-1+k:n]$ 非空）

进一步地，后缀 $s[i:n]$ 可以表示为 $k + s[i+k:n]$ ，存在后缀 $s[i+k:n]$ 。

因为后缀 $s[i+k:n]$ 在大小关系的排名上仅比后缀 $s[i-1+k:n]$ 也就是后缀 $s[i-1:n]$ 小一位，而 $h_{i-1} = \text{LCP}(s[i-1:n], s[i-2:n])$ 。

所以 $s[i:n]$ 和 $s[i-1:n]$ 有公共前缀 k 。

于是就可以得出 $h_i \geq h_{i-1} - 1$ ，也即 $h_i \geq h_{i-1} - 1$ 。

$O(n)$ 求 height 数组的代码实现

利用上面这个引理暴力求即可：

```
1 for (i = 1, k = 0; i <= n; ++i) {
2   if (rk[i] == 0) continue;
3   if (k) --k;
4   while (s[i + k] == s[sa[rk[i] - 1] + k]) ++k;
5   height[rk[i]] = k;
6 }
```

不会超过 n ，最多减 n 次，所以最多加 n 次，总复杂度就是 $O(n)$ 。

height 数组的应用

两子串最长公共前缀

感性理解：如果 h_i 一直大于某个数，前这么多位就一直没变过；反之，由于后缀已经排好序了，不可能变了之后变回来。

严格证明可以参考[\[2004\] 后缀数组 by 许智磊](#)。

有了这个定理，求两子串最长公共前缀就转化为了 [RMQ 问题](#)。

比较一个字符串的两个子串的大小关系

假设需要比较的是 $s[l_1:r_1]$ 和 $s[l_2:r_2]$ 的大小关系。

若 $h_{l_1} > r_2 - l_2 + 1$ ，

否则，。

不同子串的数目

子串就是后缀的前缀，所以可以枚举每个后缀，计算前缀总数，再减掉重复。

「前缀总数」其实就是子串个数，为。

如果按后缀排序的顺序枚举后缀，每次新增的子串就是除了与上一个后缀的 LCP 剩下的前缀。这些前缀一定是新增的，否则会破坏 的性质。只有这些前缀是新增的，因为 LCP 部分在枚举上一个前缀时计算过了。

所以答案为：

出现至少 k 次的子串的最大长度

例题：[「USACO06DEC」Milk Patterns。](#)

- 题解
- 参考代码

是否有某字符串在文本串中至少不重叠地出现了两次

可以二分目标串的长度，将 数组划分成若干个连续 LCP 大于等于 的段，利用 RMQ 对每个段求其中出现的数中最大和最小的下标，若这两个下标的距离满足条件，则一定有长度为 的字符串不重叠地出现了两次。

连续的若干个相同子串

我们可以枚举连续串的长度，按照 对整个串进行分块，对相邻两块的块首进行 LCP 与 LCS 查询，具体可见[\[2009\] 后缀数组——处理字符串的有力工具。](#)

例题：[「NOI2016」优秀的拆分。](#)

结合并查集

某些题目求解时要求你将后缀数组划分成若干个连续 LCP 长度大于等于某一值的段，亦即将 数组划分成若干个连续最小值大于等于某一值的段并统计每一段的答案。如果有多次询问，我们可以将询问离线。观察到当给定值单调递减的时候，满足条件的区间个数总是越来越少，而新区间都是两个或多个原区间相连所得，且新区间中不包含在原区间内的部分的 值都为减少到的这个值。我们只需要维护一个并查集，每次合并相邻的两个区间，并维护统计信息即可。

经典题目：[「NOI2015」品酒大会](#)

结合线段树

某些题目让你求满足条件的前若干个，而这些数又在后缀排序中的一个区间内。这时我们可以用归并排序的性质来合并两个结点的信息，利用线段树维护和查询区间答案。

结合单调栈

例题：[「AHOI2013」差异](#)

- 题解
- 参考代码

类似的题目：[「HAOI2016」找相同字符。](#)

习题

- [UVa 760 - DNA Sequencing](#)
- [UVa 1223 - Editor](#)
- [Codechef - Tandem](#)
- [Codechef - Substrings and Repetitions](#)
- [Codechef - Entangled Strings](#)
- [Codeforces - Martian Strings](#)
- [Codeforces - Little Elephant and Strings](#)
- [SPOJ - Ada and Terramorphing](#)
- [SPOJ - Ada and Substring](#)
- [UVa - 1227 - The longest constant gene](#)
- [SPOJ - Longest Common Substring](#)
- [UVa 11512 - GATTACA](#)
- [LA 7502 - Suffixes and Palindromes](#)
- [GYM - Por Costel and the Censorship Committee](#)
- [UVa 1254 - Top 10](#)

- [UVa 12191 - File Recover](#)
- [UVa 12206 - Stammering Aliens](#)
- [Codechef - Jarvis and LCP](#)
- [LA 3943 - Liking's Letter](#)
- [UVa 11107 - Life Forms](#)
- [UVa 12974 - Exquisite Strings](#)
- [UVa 10526 - Intellectual Property](#)
- [UVa 12338 - Anti-Rhyme Pairs](#)
- [DevSkills Reconstructing Blue Print of Life](#)
- [UVa 12191 - File Recover](#)
- [SPOJ - Suffix Array](#)
- [LA 4513 - Stammering Aliens](#)
- [SPOJ - LCS2](#)
- [Codeforces - Fake News \(hard\)](#)
- [SPOJ - Longest Common Substring](#)
- [SPOJ - Lexicographical Substring Search](#)
- [Codeforces - Forbidden Indices](#)
- [Codeforces - Tricky and Clever Password](#)
- [LA 6856 - Circle of digits](#)

参考资料

本页面中（[4070a9b](#) 引入的部分）主要译自博文 [Суффиксный массив](#) 与其英文翻译版 [Suffix Array](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

论文：

1. [\[2004\] 后缀数组 by. 许智磊](#)
2. [\[2009\] 后缀数组——处理字符串的有力工具 by. 罗穗骥](#)

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Chrogeek](#), [ksyx](#), [billchenchina](#), [CCXXI](#), [ChungZH](#), [Clouder0](#), [countercurrent-time](#), [Early0v0](#), [Eletary](#), [Enter-tainer](#), [gavinliu266](#), [gi-b716](#), [Great-designer](#), [Henry-ZHR](#), [HeRaNO](#), [hly1204](#), [hsfzLZH1](#), [Hukeqing](#), [iamtwz](#), [ImpleLee](#), [interestingLSY](#), [Ir1d](#), [Joob1n](#), [kenlig](#), [LingeZ3z](#), [Marcythm](#), [Menci](#), [minghu6](#), [NachtgeistW](#), [Nanarikom](#), [ouuan](#), [renbaoshuo](#), [SukkaW](#), [Tiphereth-A](#), [TOMWT-qwq](#), [WAAutoMaton](#), [werner-yin](#), [yafngzh](#), [yji9903](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用